MODULE-I

Introduction to Data Structures and Algorithms

Basic Concepts

Definition and Importance of Data Structures

- **Define Data**

  Data is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values.

  Ex : student’s name and its id are the data about the student.

- **Define Data Structure**

  Data Structure can be defined as the group of data elements which provides an efficient way of storing and organising data in the computer so that it can be used efficiently.

  OR

  Data can be organised in many different ways. The logical or mathematical model of a organisation of data is called a Data Structure.

  Ex : arrays, Linked List, Stack, Queue, etc.

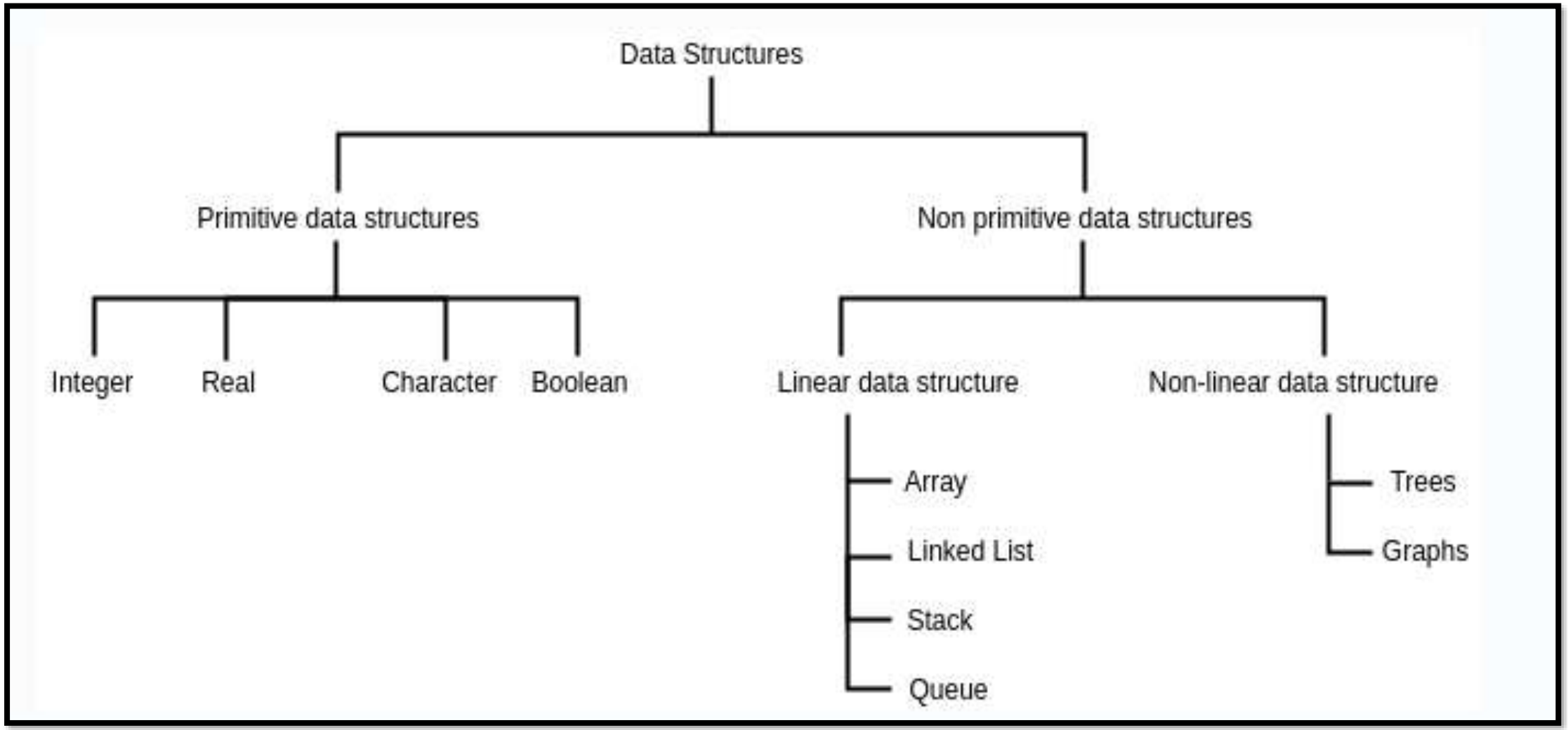
Basic Terminology

- **Data:** Data can be defined as an elementary value or the collection of values, for example, student's name and its id are the data about the student.
- **Group Items:** Data items which have subordinate data items are called Group item, for example, name of a student can have first name and the last name.
- **Record:** Record can be defined as the collection of various data items, for example, if we talk about the student entity, then its name, address, course and marks can be grouped together to form the record for the student.
- **File:** A File is a collection of various records of one type of entity, for example, if there are 60 employees in the class, then there will be 20 records in the related file where each record contains the data about each employee.
- **Attribute and Entity:** An entity represents the class of certain objects. it contains various attributes. Each attribute represents the particular property of that entity.
- **Field:** Field is a single elementary unit of information representing the attribute of an entity

Characteristics/features of data structures:

- **Linear or non-linear.** This characteristic describes whether the data items are arranged in sequential order, such as with an array, or in an unordered sequence, such as with a graph.
- **Homogeneous or heterogeneous :** This characteristic describes whether all data items in a given repository are of the same type. One example is a collection of elements in an array, or of various types, such as an abstract data type defined as a structure in C or a class specification in Java.
- **Static or dynamic :** This characteristic describes how the data structures are compiled. Static data structures have fixed sizes, structures and memory locations at compile time. Dynamic data structures have sizes, structures and memory locations that can shrink or expand, depending on the use.

Classification:



Primitive Data Structures

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like **Integer, Float, Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

1. Integer Types:

- **int** : Used to store whole numbers (integers). The size of an int depends on the architecture of the computer but is typically 4 bytes on most modern systems.
- **Ex :int age = 25;**

2. Character Type:

- **char** : Used to store a single character. It can also be used to represent small integers since each character is internally represented as a number using ASCII or a similar encoding.
- **Ex : char grade = 'A';**

3. Floating-Point Types:

- **float** : Used to store single-precision floating-point numbers, which can represent real numbers with fractional parts.
- **double** : Used to store double-precision floating-point numbers, providing greater precision compared to float.
- **Ex :**

  float pi = 3.14159;

  double temperature = 98.6;

4 . Boolean Type :

C does not have a built-in boolean data type, but you can use integer values to represent boolean values where 0 represents false and any non-zero value represents true.

- Example of boolean-like values in C:

```
// Using integer values (0 for false, 1 for true)
int isStudent = 1;
int isActive = 0;

// Using boolean macros or constants
#define true 1
#define false 0

bool isEmployed = true;
bool hasAccount = false;
```

stdbool.h and the bool data type along with the true and false constants for improved code readability:

```
#include <stdbool.h>
bool isStudent = true;
bool isActive = false;
```

Non-Primitive Data Structures

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
4. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -

   A. Linear Data Structures

   a. Static Data Structures: The data structures having a fixed size are known as Static Data Structures. Ex : Array

   b. Dynamic Data Structures: The data structures having a dynamic size are known as Dynamic Data Structures. Ex : Stack,Queue

   B. Non-Linear Data Structures

Linear Data Structures Are classified into:

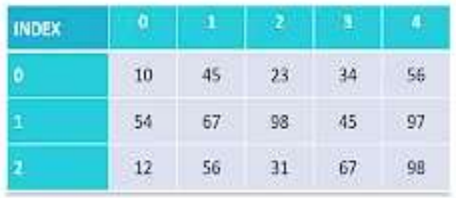
1. Arrays:

- An **array** is **defined** as the collection of similar type of data items stored at contiguous memory locations.

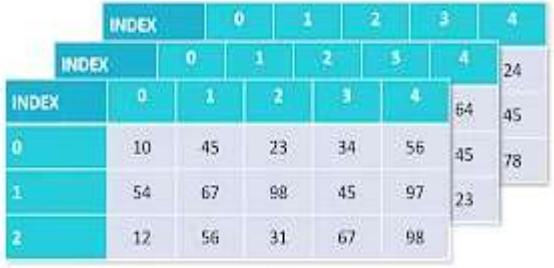
Arrays can be classified into different types:

- **One-Dimensional Array:** An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.
- **Two-Dimensional Array:** An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.
- **Multidimensional Array:** We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need.

One-Dimensional Array



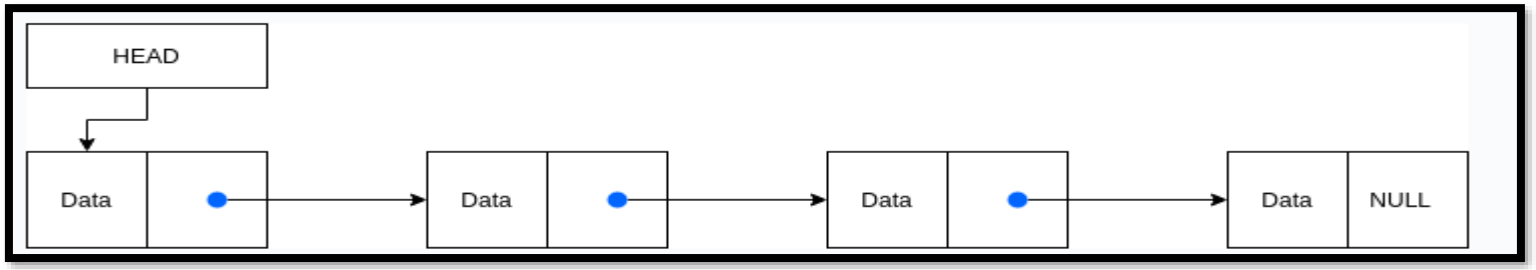
Two-Dimensional Array



Multi-Dimensional Array

2. Linked lists

- Linked list or single linked list is a sequence of elements in which every element has link to its next element in the sequence.
- Every element is called as a "node". Every "node" contains two fields, data and link. The data is a value or string and link is an address of next node.
- The first node is called HEAD which is an empty node contains an address of the first node so it link to the first node.
- The first node link to the second node and so on.The last node does not link to address but link to NULL.

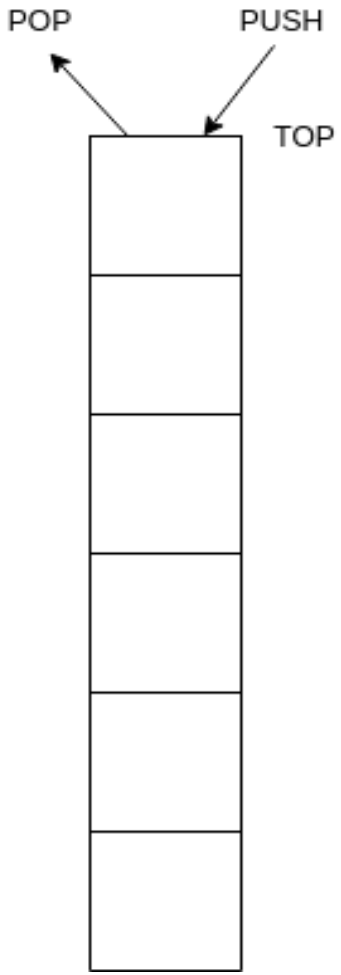


Linked Lists can be classified into different types:

- **Singly Linked List:** Each node has data and a pointer field containing an address to the next node.
- **Doubly Linked List:** A Doubly Linked List consists of an information field and two pointer fields.
- **Circular Linked List:** The last node contains the address of the first node, forming a circular loop in the Circular Linked List.

3. Stacks

- A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top.



The primary operations in the Stack are as follows:

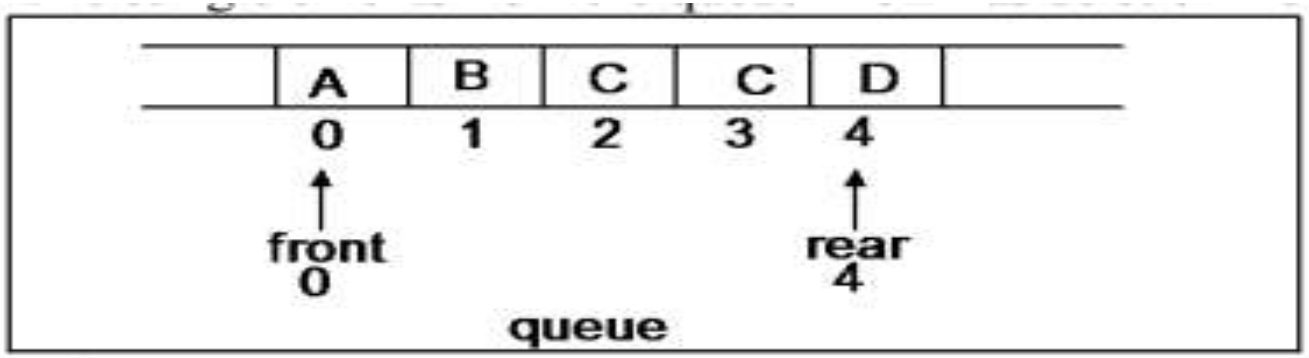
- **Push:** Operation to insert a new element in the Stack is termed as Push Operation.
- **Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.

Applications :

- Stacks are helpful in solving problems based on backtracking.
- We can use Stacks in depth-first search in graph and tree traversal.
- Stacks are also used in Operating System functions.
- Stacks are also used in UNDO and REDO functions in an edit.

4. Queues

- A queue is a FIFO(First In First Out) system, in which insertion(ENQUEUE) can take place only at an end called REAR and deletion(DEQUEUE) can take place only at another end called FRONT.



The following are the primary operations of the Queue:

- **Enqueue:** The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.
- **Dequeue:** Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.

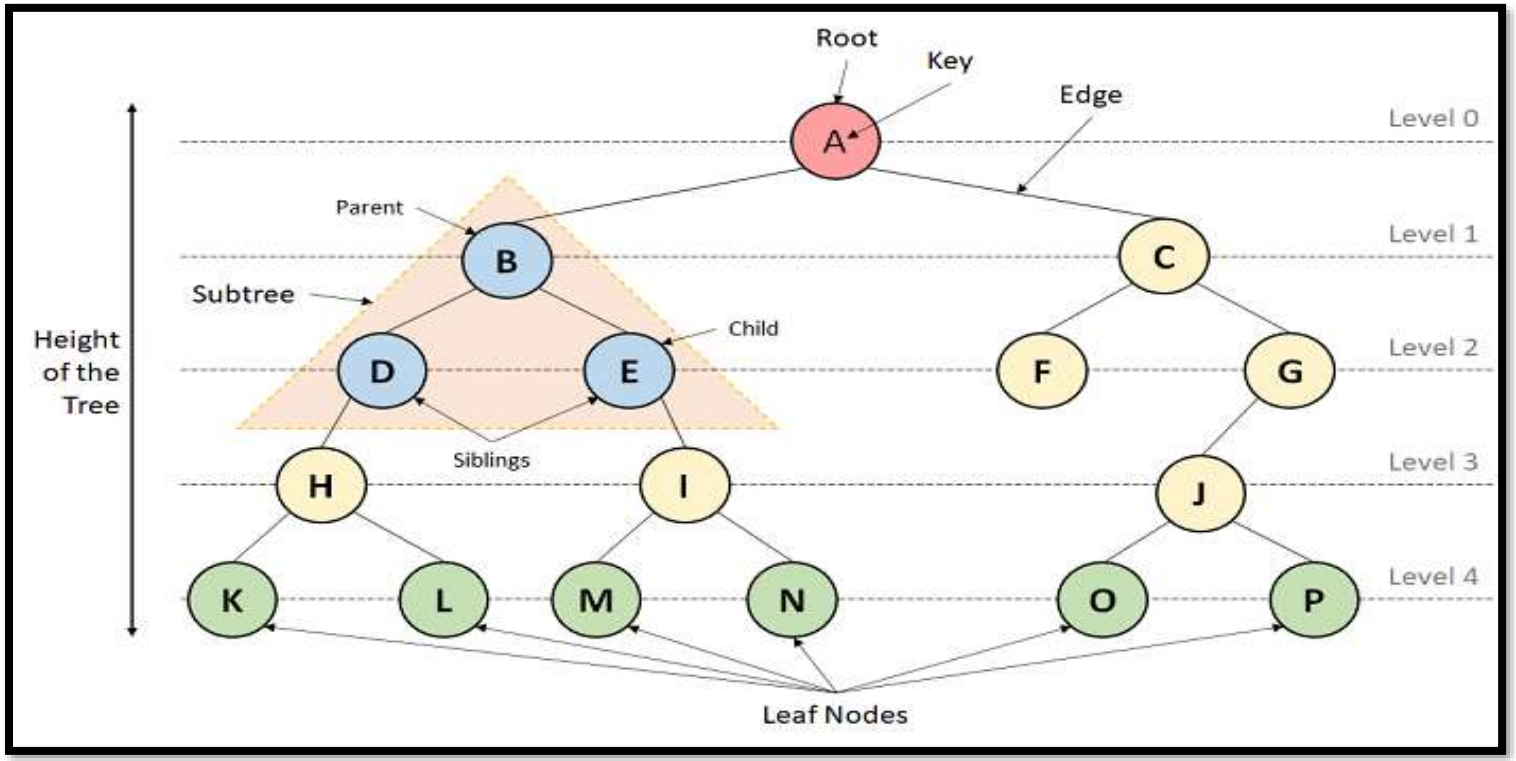
Applications :

- Queues are generally used in the breadth search operation in Graphs.
- Queues are responsible for CPU scheduling, Job scheduling, and Disk Scheduling.

Non-Linear Data Structures Are classified into:

1.Trees:

- A tree stores a collection of items in an abstract, hierarchical way. Each node is associated with a key value, with parent nodes linked to child nodes -- or subnodes. There is one root node that is the ancestor of all the nodes in the tree.



Trees can be classified into different types:

- **Binary Tree:** A Tree data structure where each parent node can have at most two children is termed a Binary Tree.
- **Binary Search Tree:** A Binary Search Tree is a Tree data structure where we can easily maintain a sorted list of numbers.
- **AVL Tree:** An AVL Tree is a self-balancing Binary Search Tree where each node maintains extra information known as a Balance Factor whose value is either -1, 0, or +1.
- **B-Tree:** A B-Tree is a special type of self-balancing Binary Search Tree where each node consists of multiple keys and can have more than two children.

Some Applications of Trees:

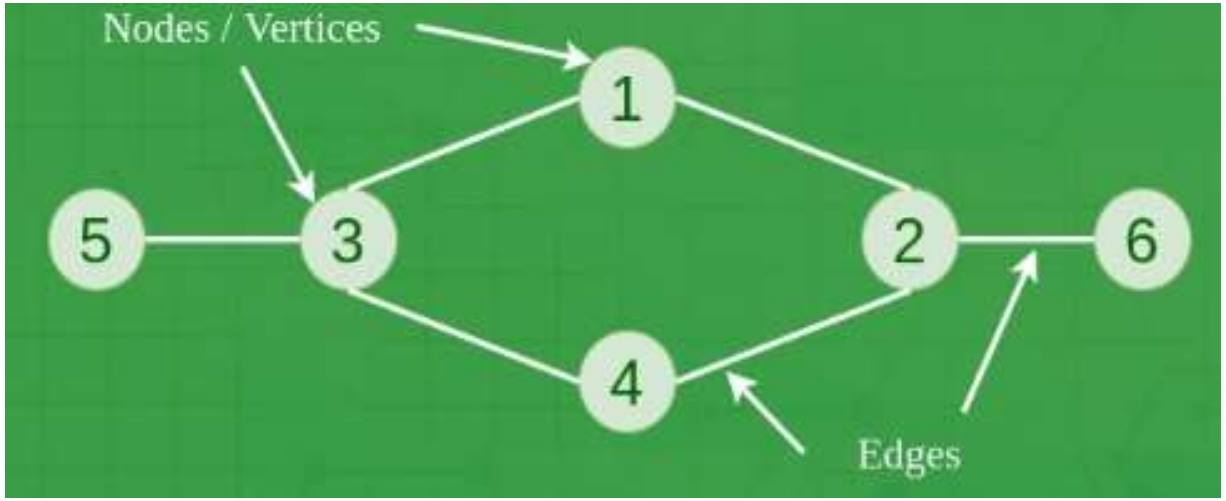
- Trees implement hierarchical structures in computer systems like directories and file systems.
- Trees are also used to implement the navigation structure of a website.
- Trees are also helpful in decision-making in Gaming applications.

2. Graphs:

- A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G(E, V).

<u>Components of a Graph</u>

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labeled/unlabelled.



<u>Graphs can be classified into different types:</u>

1. **Null Graph:** A Graph with an empty set of edges is termed a Null Graph.
2. **Simple Graph:** A Graph with neither self-loops nor multiple edges is known as a Simple Graph.
3. **Non-Directed Graph:** A Graph consisting of non-directed edges is known as a Non-Directed Graph.
4. **Directed Graph:** A Graph consisting of the directed edges between the vertices is known as a Directed Graph.
5. **Connected Graph:** A Graph with at least a single path between every pair of vertices is termed a Connected Graph.
6. **Disconnected Graph:** A Graph where there does not exist any path between at least one pair of vertices is termed a Disconnected Graph.
7. **Regular Graph:** A Graph where all vertices have the same degree is termed a Regular Graph.
8. **Complete Graph:** A Graph in which all vertices have an edge between every pair of vertices is known as a Complete Graph.

Applications of Graphs:

- Graphs are used to display routes in GPS.
- Graphs also help us represent the interconnections in social networks and other network-based applications.
- Graphs are utilized in mapping applications.

Importance of Data Structures

1. Efficient Data Storage
    1. Helps in storing large amounts of data in an organized way.
    2. Example: Arrays store multiple student marks in a single variable instead of using many variables.
2. Fast Data Access
    1. Makes searching and retrieving data quicker.
    2. Example: Binary Search Tree (BST) allows you to find a number in less time compared to a list.
3. Better Memory Management
    1. Avoids wastage of memory and uses it wisely.
    2. Example: Linked List allocates memory as needed, unlike arrays which use fixed memory.
4. Easy Data Processing
    1. Allows easy insertion, deletion, and modification of data.
    2. Example: Queue helps in processing customer requests in order (like ticket booking).

5. Helps in Solving Complex Problems
    1. Many real-life problems can be solved only with proper data structures.
    2. Example: Graphs are used in Google Maps to find the shortest route between places.
6. Supports Efficient Algorithms
    1. Some algorithms require specific data structures to work efficiently.
    2. Example: Dijkstra’s algorithm needs a priority queue (heap) for fast shortest path calculation.
7. Used in Real-Time Applications
    1. Many software and systems use data structures internally.
    2. Example: Stack is used in web browsers to go back to the previous page (undo operation).

Abstract Data Types (ADTs)

The abstract data type(ADT):

- **Abstraction** is a technique of hiding the internal details from the user and only showing the necessary details to the user.
- An **Abstract Data Type(ADT)** is a colllection of values and a set of operatios without specifying its implentation.
- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.

Ex:

```
class Smartphone
{
        private:
                int RAMSize;
                float screensize;
        public:
                void call();
                void text();
}
```

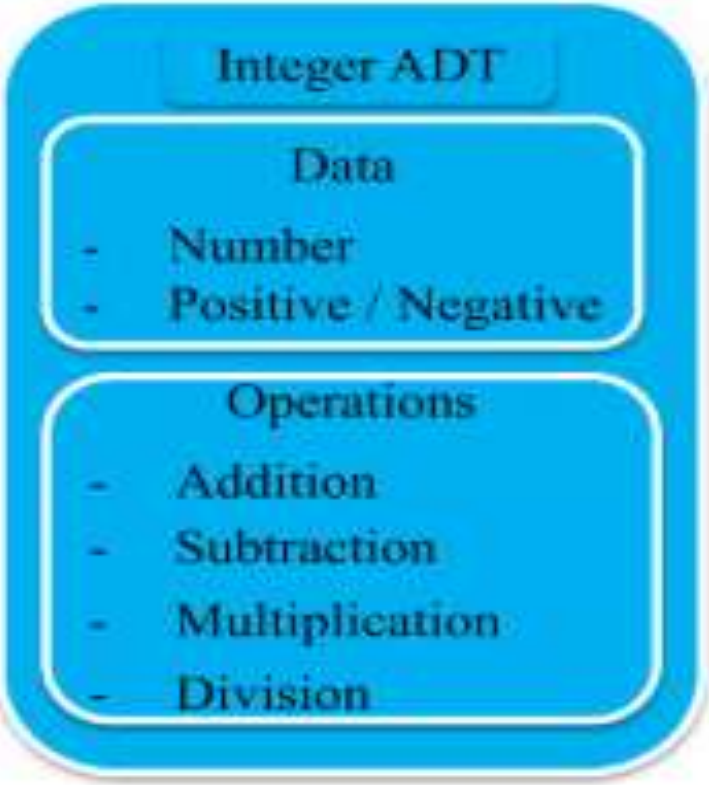
implementation view can differ

abstract/logical structure would remain same

ADT = Data + Operations

- **Simple ADTs**
  - Predefined data types are Simple ADTs.
  - **Examples:** int, float, double, char, bool
- **Consider Integer ADT:**
  - This type of ADT is an integer with predefined ranges.
  - several operations that can be applied to this data type (**addition, subtraction, multiplication, division and so on**)

Integer ADT
….. -3,-2,-1,0,1,2,3….



- **Complex ADT:**
- While several simple ADTs have been implemented, such as integer, real, character, pointer and so on, and are available for use in most languages, many useful complex ADTs are not.
- We need some **additional ADTs** that should be created and stored in the library of the computer to be used.
- **Example:**
  - **List, Set, Graph, Stack, Queue,etc.,**

Simple ADTs

Integer ADT

Data

- Number
- Positive / Negative

Operations

- Addition
- Subtraction
- Multiplication
- Division

Complex ADTs

List ADT

Data

Sequence of elements

Operations

Insert

- Find
- Delete
- Next
- Previous
- Make Empty
- Print list

Algorithm Analysis: Time and Space Complexity, Big O notation

What is Algorithm Analysis?

- When you write a program, you want it to be:
  - Fast 🏃 (Time Efficient)
  - Lightweight (Uses less memory)
- So, algorithm analysis means measuring how good or efficient an algorithm is in terms of:
  - Time Complexity (How fast it runs)
  - Space Complexity (How much memory it uses)
- We use Big O Notation to describe this.

Cntnnnn.....

- Many times there are more than one ways to solve a problem with different algorithms and we need a way to compare multiple ways.
- Also, there are situations where we would like to know how much time and resources an algorithm might take when implemented.
- To measure performance of algorithms, we typically use time and space complexity analysis.

What is Big O Notation?

- Big O notation is a mathematical way to describe the upper bound (worst-case performance) of an algorithm.
- Big O notation is a way to describe how efficient an algorithm is.
  - It shows how the time or memory increases as the size of the input increases.
  - It helps you compare algorithms without actually running them.
- It helps you:
  - Predict performance
  - Choose the best algorithm for large inputs
  - Avoid slow code as data grows

📈 Big O Growth Chart (from Fast to Slow)

Big O Notation	Name	Description
O(1)	Constant	Fast! Doesn't change with input size.
O(log n)	Logarithmic	Pretty fast. Each step cuts the work in half.
O(n)	Linear	Slower. Work grows with input size.
O(n log n)	Linearithmic	Sorting algorithms like Merge Sort.
$O(n^2)$	Quadratic	Nested loops. Very slow for large input.
$O(2^n)$	Exponential	Extremely slow. Input size doubles the work.
O(n!)	Factorial	Super slow. Try all possible combinations.

Time Complexity – Definition

- Time Complexity is the measure of the amount of time an algorithm takes to run as a function of the input size (n).
- It counts the number of basic operations or steps performed by the algorithm.

Time Complexity Examples:

```
// O(1) – Constant time
int getFirstElement(int arr[]) {
    return arr[0];
}

// O(n) – Linear time
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

// O(n^2) – Quadratic time
void printPairs(int arr[], int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            printf("(%d,%d)\n", arr[i], arr[j]);
}
```

Space Complexity – Definition

- Space Complexity is the measure of the amount of memory an algorithm uses during its execution, based on the input size (n).
- It includes memory for:
  - Variables
  - Data structures
  - Function calls (like recursion)

Space Complexity Examples

```
// O(1) – Constant space
int sum(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++)
        total += arr[i];
    return total;
}

// O(n) – Linear space (creating a new array)
int* copyArray(int arr[], int n) {
    int* copy = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++)
        copy[i] = arr[i];
    return copy;
}
```

Arrays: Definition and Operations: Insertion, Deletion, Traversal,

Definition of Array:

- An array is a collection of elements of the same data type, stored in contiguous memory locations. Each element in an array is accessed by its index or subscript. The first element is typically at index 0.

Example:

- int arr[5]; // An array of integers with 5 elements

Types of Arrays:

- One-dimensional Array (Linear Array):
  - It is a list of elements that are stored in a single row.
  - Example: int arr[5] = {1, 2, 3, 4, 5};
- Two-dimensional Array (Matrix):
  - It is an array of arrays (rows and columns).
  - Example: int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
- Multidimensional Arrays:
  - These arrays have more than two dimensions. For instance, a 3D array.
  - Example: int arr[2][3][4];

Insertion:

- Insertion refers to adding an element at a specific position in an array.
- For an ordered array, elements may need to be shifted to make space for the new element.
- Insertion can be at the beginning, end, or middle of the array.

- **Example:**

```
for(int i = n - 1; i >= pos; i--) {
    arr[i + 1] = arr[i];
}
arr[pos] = new_element;
n++;
```

Deletion:

Deletion involves removing an element from a specific position in an array. This requires shifting the remaining elements to fill the gap.

Example:

```
for(int i = pos; i < n - 1; i++) {
    arr[i] = arr[i + 1];
}
n--;
```

Traversal:

Traversal means visiting each element in the array. This is generally done using a loop to print or process the array.

Example:

```
void traverse(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

Multidimensional Arrays

Introduction to Multidimensional Arrays

- A multidimensional array is an array of arrays.
- Most commonly used: 2D arrays (matrices).
- Can be extended to 3D, 4D, etc.
- Elements are stored in contiguous memory.
- Accessed using multiple indices.
- **Syntax in C:**

```
data_type array_name[size1][size2]...[sizeN];
```

Two-Dimensional Arrays (2D Arrays)

Think of a 2D array as a table with rows and columns.

Declaration:

int matrix[3][4]; // 3 rows and 4 columns

Access: matrix[i][j] where i is row index, j is column index.

Example Initialization:

int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };

Memory Representation

- Formula to Compute Address of Element a[i][j]

  $$LOC(a[i][j])=Base(a)+w\times(i\times N+j)$$

- Where:
  - LOC(a[i][j]) = address of element at row i, column j
  - Base(a) = base address of the array (address of a[0][0])
  - w = size (in bytes) of each element (e.g., 4 bytes for int)
  - i = row index
  - j = column index
  - N = total number of columns in the array

Example:

```
int a[3][4] = {
    {10, 20, 30, 40},
    {50, 60, 70, 80},
    {90, 100, 110, 120}
};
```

Let:

- Base(a) = 1000 (assume)
- w = 4 bytes (since type is int)
- N = 4 (total columns)
- Find the address of a[2][3]:

  $LOC(a[2][3])=1000+4\times(2\times4+3)$

  $=1000+4\times(8+3)=1000+4\times11=1000+44= 1044$
- So, a[2][3] (i.e., the value 120) is stored at address 1044.

Element	Index	Formula i×N + j
a[0][0]	0×4 + 0 = 0	1000 + 4×0 = 1000
a[0][1]	0×4 + 1 = 1	1000 + 4×1 = 1004
a[0][2]	0×4 + 2 = 2	1000 + 4×2 = 1008
a[1][0]	1×4 + 0 = 4	1000 + 4×4 = 1016
a[2][3]	2×4 + 3 =11	1000 + 4×11 = 1044

Applications of Arrays

Applications of Arrays:

- **Data Storage:**

  Arrays are often used to store data of the same type, such as lists of integers, floats, or characters. They are used in applications that require indexing like databases and file handling.

- **Matrix Operations:**

  Arrays are fundamental for operations on matrices in mathematical computations, graphics, and physics simulations.

- **Sorting Algorithms:**

  Arrays are used to implement various sorting algorithms like Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, etc.

- **Searching Algorithms:**

  Arrays help in searching elements using techniques like Linear Search and Binary Search.

- **Buffer Storage:**

  Arrays are used as buffers in many applications, such as network data buffering, image processing, and more.

- **Game Development:**

  Arrays are often used in game development to represent grids, maps, and scores.

- **Data Representation:**

  Arrays are commonly used in computer science for representing data structures like heaps, hash tables, and queues.

Linked Lists:

Singly Linked List: Creation, Insertion, Deletion, Traversal

DEFINITION

- A linked list, or one-way list, is a linear collection of data elements, called nodes, where the linear order is given by means of pointers. That is, each node is divided into two parts:

1) The first part contains the information of the element, and

2) The second part, called the ***link field*** **or** ***nextpointer***field, contains the address of the next node in the list.

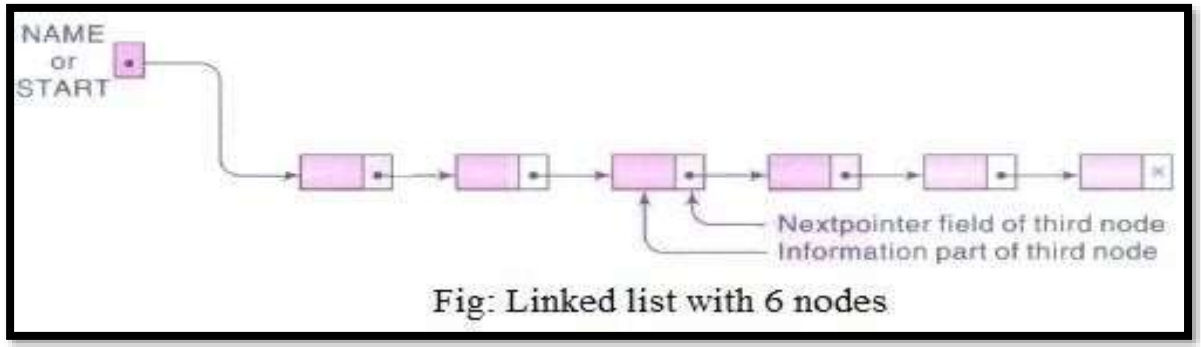


Fig: Linked list with 6 nodes

<u>In the above figure each node is pictured with two parts.</u>

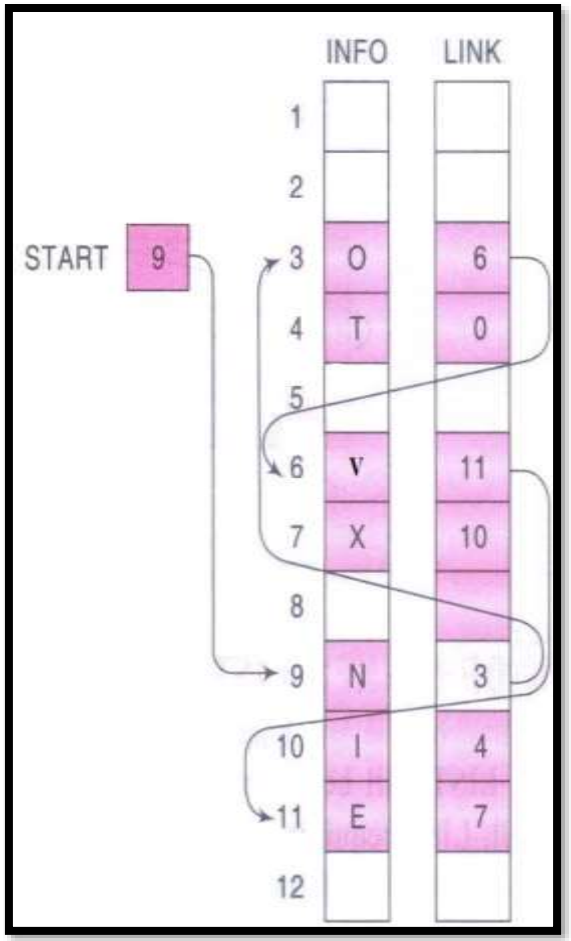
- The left part represents the information part of the node, which may contain an entire record of data items.
- The right part represents the nextpointer field of the node
- An arrow drawn from a node to the next node in the list.
- The pointer of the last node contains a special value, called the *null pointer*, which is any invalid address.

- A pointer variable called **START or FIRST** which contains the address of the first node. A special case is the list that has no nodes, such a list is called the ***null list or empty list*** and is denoted by the null pointer in the variable START.

REPRESENTATION OF LINKED LISTS IN MEMORY

- Let LIST be a linked list. Then LIST will be maintained in memory as follows.
- LIST requires two linear arrays such as INFO and LINK-such that INFO[K] and LINK[K] contains the information part and the nextpointer field of a node of LIST.
- LIST also requires a variable name such as START which contains the location of the beginning of the list, and a nextpointer sentinel denoted by NULL-which indicates the end of the list.
- The subscripts of the arrays INFO and LINK will be positive, so choose NULL = 0, unless otherwise stated.

The following examples of linked lists indicate that the nodes of a list need not occupy adjacent elements in the arrays INFO and LINK, and that more than one list may be maintained in the same linear arrays INFO and LINK. However, each list must have its own pointer variable giving the location of its first node.



- START=9
- INFO[9]=N
- LINK[3]=6
- INFO[6]=V
- LINK[6]=11
- INFO[11]=E
- LINK[11]=7
- INFO[7]= X
- LINK[7]=10
- INFO[10]= I
- LINK[10]=4
- INFO[4]= T
- LINK[4]= NULL value, So the list has ended

REPRESENTING CHAIN IN C

The following capabilities are needed to make linked representation

- A mechanism for defining a node's structure, that is, the field it contains. So selfreferential structures can be used
- A way to create new nodes, so MALLOC functions can do this operation
- A way to remove nodes that no longer needed. The FREE function handles this operation.

Defining a node structure

```
 typedef struct listNode *listPointer;
 typedef struct {
            char data[4];
   listPointer list;
} listNode;
```

Create a New Empty list

```
listPointer first = NULL
```

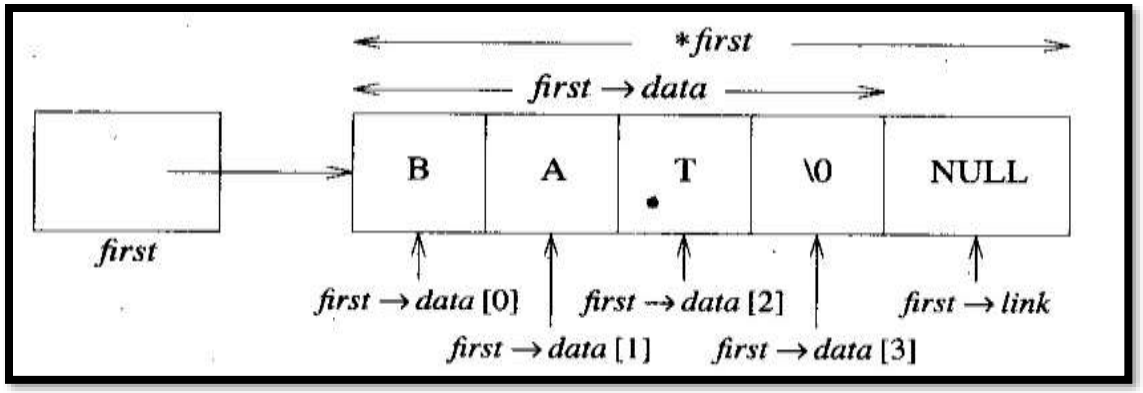
To create a New Node

```
MALLOC (first, sizeof(*first));
```

To place the data into NODE

strcpy(first→ data,”BAT”);

first→ link = NULL



The below steps should be followed to insert a new node at the front of the linked list

1) Allocate a new node (say **temp**).
2) Put the required data into **temp**.
3) The 'next' pointer of the node should be pointed to the current **head**.
4) Now make the **head** pointer point to **temp**.

```
Void insert(listPointe first,listPointer x)
{/*insert a new node with data=50 into the chain first after node x*/
listPointer temp;
MALLOC(temp,sizeof(*temp));
temp->data=50;
if(*first){
    temp->link=x->link;
    x->link=temp;
}
else{
    temp->link=NULL;
    *first=temp;
}
}
```

To delete a node from the linked list, we need to do the following steps.

1) Find the previous node of the node to be deleted.
2) Change the next of the previous node.
3) Free memory for the node to be deleted.

```
void delete(listPointer *first,listPointer trail,listPointer x)
{*delete x from the list,trail is the preceding node and *first is the front of the list*/
if(trial)
	trial->link=x->link;
else
	*first=(*first)->link;
free(x);
}
```

Traversal

```
void traverse(listPointer first) {
    listPointer temp = first;  // Start from the head

    while (temp != NULL) {
        printf("%d -> ", temp->data);  // Print data
        temp = temp->link;            // Move to the next node
    }

    printf("NULL\n");  // End of the list
}
```

Doubly Linked List

DOUBLY LINKED LIST

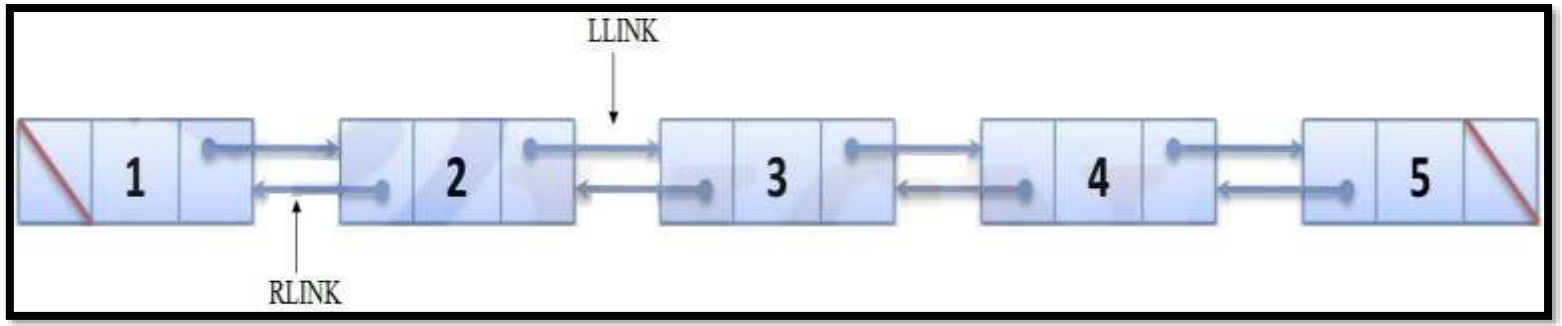
- The difficulties with single linked lists is that, it is possible to traversal only in one direction, ie., direction of the links.
- The only way to find the node that precedes p is to start at the beginning of the list. The same problem arises when one wishes to delete an arbitrary node from a singly linked list. Hence the solution is to use doubly linked list

Doubly linked list: It is a linear collection of data elements, called nodes, where each node N is divided into three parts:

- An information field INFO which contains the data of N
- A pointer field LLINK (FORW) which contains the location of the preceding node in the list
- A pointer field RLINK (BACK) which contains the location of the in the next node list

The declarations are:

```
typedef struct node *nodePointer;
typedef struct {
        nodePointerllink;
        element data;
        nodePointerrlink;
} node;
```



Insertion into a doubly linked list

- Insertion into a doubly linked list is fairly easy. Assume there are two nodes, node and newnode, node may be either a header node or an interior node in a list. The function dinsert performs the insertion operation in constant time.
- Program: **Insertion** into a doubly linked circular list

```
void dinsert(nodePointer node, nodePointernewnode)
{/* insert newnode to the right of node */
        newnode→llink = node;
        newnode→rlink = node→rlink;
        node→rlink→llink = newnode;
        node→rlink = newnode;
}
```

Deletion from a doubly linked list

- Deletion from a doubly linked list is equally easy. The function*ddelete* deletes the node deleted from the list pointed to by node. To accomplish this deletion, we only need to change the link fields of the nodes that precede(deleted→llink→rlink) and follow (deleted→rlink→llink) the node we want to delete.
- Program: **Deletion** from a doubly linked circular list

```
void ddelete(nodePointer node, nodePointer deleted)
{/* delete from the doubly linked list */
        if (node == deleted)
        printf("Deletion of header node not permitted.\n");
        else {
                deleted→llink→rlink= deleted→rlink;
                deleted→rlink→llink = deleted→llink;
                free(deleted) ;
            }
}
```

Circular Linked List

Introduction to Circular Linked List

- A Circular Linked List (CLL) is a variation of a linked list.
- The last node points to the first node instead of NULL.
- Can be singly or doubly circular.

Types of Circular Linked Lists

- Singly Circular Linked List
  - Each node has data and next pointer.
  - Last node’s next → head.

- Doubly Circular Linked List
  - Each node has data, prev, and next.
  - Last node’s next → head
  - head->prev → last node

Operations on Circular Linked List

- Traversal
  - Start from head, move using next
  - Stop when current node is back to head
- Insertion
  - At beginning
  - At end
  - After a specific node
- Deletion
  - First node
  - Specific node
  - Last node

Advantages of Circular Linked List

- No NULL in the list
- Can traverse from any node
- Good for applications requiring circular movement (e.g., scheduling)

Limitations

- More complex to manage than linear lists
- Care must be taken to avoid infinite loops during traversal

Applications of Circular Linked List

- Round-robin scheduling (CPU/task scheduling)
- Multiplayer games (cyclic turns)
- Circular queues
- Music playlists

Applications of Linked Lists

Application Area	Description
Dynamic memory use	Nodes created at runtime
Data structures	Stack, queue, graph, hash table
Circular processes	Scheduling, playlists, buffers
Polynomial expressions	Easy term-wise operations
OS & Garbage Collection	Free lists, memory reuse
Editor/IDE features	Undo/Redo using doubly linked lists
Web/Software navigation	Forward and backward navigation
Algorithm design	Temporary storage in advanced logic

THANK YOU